Missing Manuscript Revisions

Text-only comparison against the Solis revision guide

No. Several text revisions are implemented, but the manuscript is not yet fully aligned with the Solis guide.

Reviewed: BulSUScholar_Exact_Thesis_DOC_SOLIS_Revision_Guide.pdf and CENTRALIZED-SCHOLARSHIP-MANAGEMENT-AND-TRACKING-SYSTEM-FOR-BULACAN-STATE-UNIVERSITY.docx, both in docs/ThesisPaper.

Graphical changes are excluded. Written explanations beneath figures are included because they are text. Neither source document was modified.

Substantially Completed

Guide Item	Manuscript Section	Assessment
R2	Chapter II > Duplication Prevention	Matching rationale, normalization, weights, thresholds, and similarity-versus-award distinction are explained.
R2-B	Chapter III > Duplication Prevention Algorithm	Actual duplicate rules and separate official-assignment checks are included.
R3-A	Chapter III > Approval Module	Account activation, document validation, profile review, and scholarship assignment are distinguished.
R3-B	Explanation below Figure 3.1	Manual/automatic activation and subsequent roster reconciliation are explained.
R3-C	Explanation below Figure 3.2	Automatic COR/ROG processing, manual profile review, and non-recurring identity requirements are explained.
R4	Explanation below Figure 3.5	Import validation, pending-account reconciliation, conflicts, document preservation, and administrator resolution are covered.

Partially Completed

Guide Item	What Still Needs Attention
R1-A - Scope	External grantors and imports are included, but the statement distinguishing Government, Private, and Others classifications is missing.
R1-B - Conceptual Framework	The new explanation is present, but change "referred to Administrator" to "referred to a Full Administrator."
R1-C - Limitations	External-record limitations are explained, but the paragraph still omits the implemented official roster-import process as an input method.
R5 - Monitoring	The semester/status explanations are included, but the requested six-row scholarship-status table is missing.
R6 - Testing Methodology	Targeted testing is mentioned, but the isolated PostgreSQL/PGlite environment, synthetic records, and evidence limitations are not clearly documented.

Still Missing or Not Updated

Guide Item	Required Text Revision
R6-B	Update the Table 1 discussion to distinguish the existing 15 selected cases from additional dated, verified testing activities.
R7-A	Add "Results for Specific Objective No. 2: Record Validation and Prevention of Multiple Active Scholarships" and the selected Solis test-case table.
R7-B	Replace the explanation below Figure 4.11. It still broadly says a possible duplicate restricts approval, without the required distinction between similarity warnings and confirmed scholarship conflicts.
R8-A	Add the Document Extraction Accuracy Evaluation methodology.
R8-B	Add actual extraction findings and the document-sample results table after conducting the evaluation. Do not invent results.
R9-A	Update Chapter V > Summary of Findings No. 2.
R9-B	Update Chapter V > validation conclusion.
R9-C	Update the student-verification recommendation to explicitly describe improving already-implemented capabilities.
R9-D	Update the external-integration recommendation to acknowledge existing official roster imports.
R11-A	Add the actual recommendation-code excerpt, explanation, and a concrete worked example.
R11-B	Add actual normalization, Levenshtein, weighted-comparison, duplicate-condition, and roster-assignment code excerpts with explanations.
R12-D	Add introductory paragraphs and numbered captions for the new technical-test tables once inserted.

Important Algorithm Correction

The guide itself retains outdated Decision Tree wording in R11-A. The current backend identifies the algorithm as Weighted Recommendation Scoring. The manuscript still says Decision Tree in several present-system descriptions. Correct those descriptions rather than copying that outdated guide wording.

Conditional Items

R10-A and R10-C are not applied; the manuscript still uses IPO. The guide requires DSRM replacement only if your adviser approves it, so these should not automatically be counted as unfinished mandatory revisions.

Bottom Line

The core workflow explanations are largely updated. The largest remaining work is testing evidence, extraction evaluation, Chapter V alignment, and actual algorithm-code explanations. Neither document was modified.

Detailed Manual Editing Instructions

The following entries extend the review above. Each gives the location, an exact current Word search phrase, the edit boundary, and suggested replacement or insertion text. Edit the DOCX yourself. Keep completed sections unless a specific correction is listed. Graphical figures and images are outside this guide; explanations beneath them remain included as text.

Suggested paragraphs can be adapted without changing their meaning. Do not paste labels such as WHERE or WHAT TO DO into the thesis. Source paragraph indexes are included only to trace the current DOCX; they are not Word page numbers. Table numbers must be assigned after all insertions. Evidence marked pending must not be presented as completed research.

R1-A. Complete the scope classification statement

WHERE: Chapter I > Scope and Limitations of the Study > Scope of the Study

FIND IN WORD: "External scholarship providers, including local government units"

Source body paragraph(s): 82. These are not printed page numbers.

WHAT TO DO: Replace only this External-provider paragraph. Keep the preceding centralization paragraph and the following scholarship-recommendation paragraph.

STOP BEFORE: "The system also provides scholarship recommendations"

WHAT TO PUT:

External scholarship providers, including local government units and other organizations, may be represented by grantor accounts classified as External. Authorized users can upload official scholarship recipient lists through the existing spreadsheet roster-import process. Government, Private, and Others remain separate provider classifications. External scholarship records become available for identity matching and scholarship-assignment validation after they have been entered and validated within the system.

R1-B. Specify the administrator authorized to resolve conflicts

WHERE: Chapter I > Conceptual Framework > process explanation

FIND IN WORD: "Imported scholarship rosters may also be reconciled"

Source body paragraph(s): 75. These are not printed page numbers.

WHAT TO DO: Replace this paragraph only. The important correction is changing "Administrator" to "a Full Administrator"; preserve the existing distinction among account activation, review, approval and assignment.

STOP BEFORE: "The output stage represents"

WHAT TO PUT:

Imported scholarship rosters may also be reconciled when a matching student account is activated or completes verified login. Possible identity conflicts are referred to a Full Administrator for verification. Account activation, document review, scholarship application approval, and official scholarship assignment are treated as separate operations. Once the applicable validation and approval procedures are completed, the system updates the corresponding scholarship records and supports subsequent monitoring, renewal, notification, and reporting activities.

NOTE: If the adviser approves the conditional DSRM replacement in R10-C, use that replacement for the framework explanation instead of keeping both competing framework descriptions.

R1-C. Acknowledge roster import in the limitations

WHERE: Chapter I > Scope and Limitations of the Study > Limitations of the Study

FIND IN WORD: "The system does not provide full real-time system-to-system integration"

Source body paragraph(s): 89. These are not printed page numbers.

WHAT TO DO: Replace this paragraph only. Keep the following paragraph on data accuracy.

STOP BEFORE: "The accuracy of the system depends"

WHAT TO PUT:

The system does not provide full real-time system-to-system integration with external scholarship providers such as DOST, UNIFAST, local government scholarship offices, and other government or private scholarship organizations. Externally sourced scholarship information must first be uploaded through the official roster-import process, manually encoded where supported, or maintained by authorized users.

The system can compare only scholarship information that has been entered and sufficiently updated within its centralized database. Missing, incomplete, or outdated external scholarship records may therefore limit the identification of conflicting scholarship assignments. Direct integration with external scholarship databases remains a possible future enhancement, subject to institutional authorization, data-sharing agreements, and appropriate security requirements.

R5. Add the missing scholarship-status table

WHERE: Chapter III > System Modules > 8. Monitoring Module

FIND IN WORD: "Legacy scholarship records without verified academic-cycle information"

Source body paragraph(s): 308. These are not printed page numbers.

WHAT TO DO: Keep the existing cycle/status paragraphs. Insert the following introductory paragraph, caption and six-row table after this legacy-record paragraph and before 9. Notification Module.

STOP BEFORE: "9. Notification Module"

WHAT TO PUT:

Table [assign number] summarizes how scholarship records are interpreted according to their commitment status and academic cycle. The distinction prevents the completion of a workflow from being mistaken for permission to receive another conflicting scholarship within the same cycle.

Table [assign number]. Academic-Cycle-Aware Scholarship Status Rules

Scholarship Record	Interpretation / Expected Handling
Confirmed current-cycle award	Preserve the award or commitment and prevent another conflicting current-cycle assignment.
Current-cycle award marked Finished or Completed	Workflow completion does not remove the current-cycle award restriction.
Completed award from an explicitly recorded previous cycle	Keep as history; it does not independently block an eligible current-cycle application or assignment.
Cancelled, terminated or inactive commitment	Do not treat the cancelled/inactive commitment itself as an active-award blocker. Other valid awards, account restrictions and eligibility rules still apply.
Pending application without a confirmed award or commitment	Distinguish it from an existing award. Archive competing current-cycle pending applications and release reserved slots when the applicable assignment transaction succeeds.
Legacy record without verified academic-cycle information	Refer for authorized verification; do not guess the semester or automatically create an assignment.

The Approval Module already explains archiving competing current-cycle pending applications, releasing slots and retaining historical documents. Keep that corrected wording rather than adding a second conflicting rule.

R6. Complete the technical-testing methodology

WHERE: Chapter III > Testing

FIND IN WORD: "In addition to the selected core functional tests"

Source body paragraph(s): 219. These are not printed page numbers.

WHAT TO DO: Replace the targeted-testing block beginning here, including the next three testing paragraphs. Keep the preceding functional-testing paragraph. Stop before the existing TAM/ISO assessment paragraph.

STOP BEFORE: "The developed system was also evaluated using standardized models"

WHAT TO PUT:

In addition to the selected core functional tests, targeted automated testing was conducted to verify student-record matching, official scholarship roster import, identity reconciliation, scholarship assignment, and prevention of conflicting current-cycle awards.

Backend regression tests examined exact identity matching, configured similarity thresholds, account activation and verified-login requirements, role-based authorization, external grantor classification, and scholarship conflict handling.

Focused database tests were performed using an isolated PostgreSQL-compatible PGlite environment and synthetic student and scholarship records. The tests assessed roster assignment, repeated processing, current-cycle award conflicts, previous-cycle historical records, cancelled or terminated commitments, competing pending applications, reservation release, administrative restrictions, and transaction rollback behavior.

Each test was documented using an identifiable test-case number, a description of the function being evaluated, expected results, actual observed results, and the corresponding test status. These automated technical tests were separate from the acceptability assessment using TAM and the software-quality evaluation using ISO/IEC 25010. Successful technical test execution does not independently establish document-extraction accuracy on actual student documents or the completeness of external scholarship records.

NOTE: This describes the recorded controlled tests, not a new production test. Keep the synthetic-data and environment qualifications.

R6-B. Update the Table 1 discussion without mixing test sets

WHERE: Chapter III > Testing > paragraph following Table 1

FIND IN WORD: "Table 1 presents the summary of the selected core test cases"

Source body paragraph(s): 227. These are not printed page numbers.

WHAT TO DO: Replace this Table 1 discussion only. Keep the existing 15 selected cases if their supporting evidence remains valid. The dated counts below refer to the Solis verification snapshot, not the latest total from unrelated revisions.

STOP BEFORE: "Data Gathering Instrument"

WHAT TO PUT:

Table 1 presents 15 selected core automated backend test cases conducted to verify the functionality of major system modules. The table summarizes the tested operations, their expected outcomes, actual observed results, and corresponding test status.

Additional targeted verification activities were recorded separately from the selected core test cases. The implementation verification documented on October 23, 2026, reported 348 passing backend tests, including 23 added Solis-specific regression tests. A separate focused database test process reported 27 passing

synthetic test cases. Seven frontend security checks, frontend lint verification, and the production build were also reported as successful.

These results represent different sets of technical verification activities and were not combined into a single algorithm-accuracy percentage or treated as equivalent to real-user evaluation. The results are limited to the documented testing environments and datasets.

NOTE: Historical evidence: docs/Solis_System_Revision.md; backend/tests/test_solis_system_revision.py contains 23 test methods; test-results/solis-system-cases.json contains 27 recorded passing cases. If presenting a later test run, cite its date and artifact separately rather than silently replacing this snapshot.

R7-A. Add Objective No. 2 results and selected test cases

WHERE: Chapter IV > Designing and Developing the System > Validation Module discussion

FIND IN WORD: "The Validation Module supports student-record verification"

Source body paragraph(s): 491. These are not printed page numbers.

WHAT TO DO: Keep this existing Validation Module paragraph. Immediately after it, add the following subheading, discussion and selected test table. Place them before the Approval Module paragraph.

STOP BEFORE: "The Approval Module allows authorized scholarship grantors"

WHAT TO PUT:

Results for Specific Objective No. 2: Record Validation and Prevention of Multiple Active Scholarships

The developed system's student-record verification and duplicate-scholarship prevention functions were evaluated through controlled test cases involving student identity information, internal scholarship records, and official scholarship roster records available within the system. The tests examined exact identity matching, possible duplicate-record identification, academic-cycle validation, existing scholarship commitment checks, roster reconciliation, and administrative conflict resolution.

The focused database verification demonstrated that existing current-cycle scholarship awards were preserved when competing assignments were attempted. Completed scholarship records from previous academic cycles were maintained as historical records and did not independently prevent current-cycle eligibility processing. Additional tests evaluated mismatched identities, repeated roster-import operations, competing pending applications, administrator authorization restrictions, and the preservation of submitted documents and scholarship records.

The reported results support the expected behavior of the tested implementation under controlled synthetic conditions. They do not establish complete detection of scholarships from unrecorded external providers, measured extraction accuracy using actual COR and ROG documents, or successful production deployment of the revised implementation.

Table [assign number] presents selected controlled database cases used to evaluate identity validation, academic-cycle rules and scholarship-conflict safeguards. The table identifies each case, expected behavior, actual recorded result and test status.

Table [assign number]. Selected Objective No. 2 Validation and Scholarship-Conflict Test Cases

Case ID	Core Function	Expected Result	Actual / Status
SOL-SYS-02	External roster exact assignment	Exact ID and normalized name assign one scholarship in normalized state	As expected / Pass
SOL-SYS-04	Name mismatch	A similar/different name never authorizes assignment; an idempotent warning is recorded	As expected / Pass

Case ID	Core Function	Expected Result	Actual / Status
SOL-SYS-05	Same-cycle completed scholarship	A completed current-cycle award is preserved and a second scholarship is flagged	As expected / Pass
SOL-SYS-06	Previous-cycle completed award	Historical completed progress remains unchanged and does not block a current-cycle assignment	As expected / Pass
SOL-SYS-11	Resolution authorization and reason	Reviewers and blank reasons cannot change a conflict	As expected / Pass
SOL-SYS-17	Existing scholar applies elsewhere	Current awarded student is blocked before a new application or slot decrement	As expected / Pass

Suggested discussion after the table: The selected cases demonstrate the expected behavior of the implemented safeguards for controlled records. A denied conflicting assignment is a passed test when denial is the expected result. These tests demonstrate the specified workflow rules, not the completeness of external scholarship detection or real-document extraction accuracy.

Evidence: test-results/solis-system-cases.json; recorded environment: Isolated PostgreSQL/PGlite; dataset: Synthetic fixtures; no production student records. These are historical recorded results, not tests newly executed while preparing this PDF.

R7-B. Correct the written explanation below Figure 4.11

WHERE: Chapter IV > Presentation and Description of the System > Duplicate Scholarship Validation

FIND IN WORD: "Figure 4.11 presents the duplicate scholarship validation feature"

Source body paragraph(s): 532. These are not printed page numbers.

WHAT TO DO: Replace the explanation under Figure 4.11, not the image. Stop before Figure 4.12. Remove the implication that every similarity warning universally blocks every approval operation.

STOP BEFORE: "Figure 4.12. Approval Page"

WHAT TO PUT:

Figure 4.11 presents the duplicate-scholarship validation feature of the developed system. The validation process checks available student and scholarship records to identify potential duplicate identities and conflicting scholarship commitments.

General duplicate-record checking uses exact identity comparisons and similarity-based matching to identify records that may refer to the same individual. Possible duplicate identities may be flagged for authorized review. A similarity-based warning alone does not establish that a student has already received a scholarship or authorize another scholarship assignment.

Official scholarship roster assignment uses separate checks involving student identity, recognized scholarship information, academic-cycle verification, and existing scholarship commitments. A confirmed competing scholarship award within the current academic cycle prevents an additional conflicting application or automatic assignment.

Conflicts requiring administrative intervention are referred to a Full Administrator through the corresponding warning interface, with documented resolution reasons. This process preserves existing scholarship records and helps prevent conflicting scholarship assignments using information available within the system.

R8-A. Add the extraction-evaluation method without claiming it was completed

WHERE: Chapter III > Testing > after the R6 technical-testing block and before TAM/ISO

FIND IN WORD: "Each test was documented using an identifiable test-case number"

Source body paragraph(s): 222. These are not printed page numbers.

WHAT TO DO: After completing R6, insert the following subheading and method at the end of its targeted-testing discussion, before the existing TAM/ISO paragraph. The anchor identifies the current block being replaced by R6.

STOP BEFORE: "The developed system was also evaluated using standardized models"

WHAT TO PUT:

Document Extraction Accuracy Evaluation

A separate document-extraction evaluation is proposed to assess the system's ability to identify and validate student information from Certificates of Registration (COR) and applicable Reports of Grades (ROG). The evaluation will compare extracted information against manually verified reference values from authorized document samples.

The sample set will include text-based PDFs, clear scanned PDFs, low-quality copies, documents containing easily confused characters such as 7 and 1, and documents with missing or incorrect information. The assessment will cover Student ID recognition, applicable academic information and supported grade-related fields. Expected validation rejections, including wrong-owner or wrong-cycle documents, will be documented separately from extraction correctness.

Outcomes will be recorded as correctly extracted, incorrectly extracted, or rejected/unreadable. Rejected documents will not automatically be counted as successful extractions. The sample size, selection method, document conditions and reference values will be documented. Real student information will be protected or masked in reported examples.

This evaluation is separate from duplicate-record matching and scholarship-assignment tests. Until the evaluation has been performed and documented, the study will not claim a measured extraction-accuracy percentage.

NOTE: Use this proposed-method wording while real-document evaluation is pending. After actually conducting the evaluation, replace the future/proposed wording with an accurate account of what was performed. Do not present the procedure as completed merely to satisfy the guide.

R8-B. Report extraction results only when evidence exists

WHERE: Chapter IV > the new Objective No. 2 results subsection, after the selected Solis case table

FIND IN WORD: "The Validation Module supports student-record verification"

Source body paragraph(s): 491. These are not printed page numbers.

WHAT TO DO: Use the existing Validation Module paragraph to locate R7-A first. After the selected test table and discussion added by R7-A, insert the honest pending-evidence statement below. Replace it with measured findings only after performing the evaluation.

WHAT TO PUT:

The controlled matching and scholarship-assignment tests do not measure document-extraction accuracy on actual COR and ROG documents. A separate authorized-sample evaluation remains necessary; therefore, no real-document extraction-accuracy percentage is claimed in the present results.

NOTE: This pending statement is not a completed accuracy result and does not close the evidence requirement. The sample table and completed-results template below must remain unclaimed until actual measurements are collected.

WORKSHEET TO COMPLETE AFTER TESTING: use masked reference and extracted values. These rows are test categories to carry out, not recorded outcomes. Assign a final caption only when inserting a completed table into Chapter IV.

Sample ID	Document / Condition	Values To Compare	Actual Outcome
DOC-001	Text-based COR or ROG PDF	Masked Student ID, academic cycle, and other supported fields against manually verified reference	Not tested / fill after evaluation
DOC-002	Clear scanned PDF	Same supported fields; include applicable ROG grade/GWA information	Not tested / fill after evaluation
DOC-003	Low-quality scan or 7-versus-1 case	Record exact reference and extracted characters, errors and unreadable results	Not tested / fill after evaluation
DOC-004	Wrong-owner or wrong-cycle document	Expected validation response versus actual response; distinguish rejection from extraction correctness	Not tested / fill after evaluation

COMPLETED-RESULTS TEMPLATE - DO NOT PASTE AS A FINISHED RESULT YET:

The document-extraction evaluation used [verified number] authorized COR and ROG samples representing [actual document conditions]. Extracted values were compared with manually verified reference values. Of the evaluated samples, [number] produced the expected extracted information, [number] contained extraction discrepancies, and [number] were rejected or could not be processed. The evaluation identified [actual findings]. These results describe the evaluated sample set and do not establish performance on all student documents.

R9-A. Replace Summary of Findings No. 2

WHERE: Chapter V > Summary of Findings

FIND IN WORD: "2. The system implements a pre-approval validation mechanism"

Source body paragraph(s): 601. These are not printed page numbers.

WHAT TO DO: Replace Finding No. 2 only. Keep the other findings and survey statistics unchanged.

WHAT TO PUT:

2. The developed system implements student-record verification, duplicate-record detection, and academic-cycle-aware scholarship-assignment safeguards using available internal scholarship information and validated external scholarship rosters stored within the centralized database. The validation mechanism employs exact identity checks, weighted similarity comparisons, and scholarship commitment validation to identify potential duplicates and prevent conflicting awards within the applicable academic cycle. Controlled technical tests support the implemented validation behavior under the evaluated conditions. The completeness of external scholarship detection remains dependent on the availability and accuracy of records entered into the system.

R9-B. Replace the validation conclusion

WHERE: Chapter V > Conclusions

FIND IN WORD: "The pre-approval validation mechanism supports the prevention"

Source body paragraph(s): 608. These are not printed page numbers.

WHAT TO DO: Replace only this validation conclusion. Keep the other conclusions and the TAM/ISO results.

WHAT TO PUT:

The implemented validation and scholarship-assignment mechanisms support the prevention of conflicting scholarship awards through student-record verification, duplicate-detection procedures, and academic-cycle-aware commitment checks. The system distinguishes possible duplicate identities from confirmed scholarship assignments and preserves existing current-cycle awards when conflicting assignments are attempted. Previous-cycle scholarship records are retained for monitoring and historical reference.

These mechanisms strengthen the management of scholarship records available within the centralized database. The system does not automatically identify scholarships that have not been entered or updated by authorized users, and similarity-based matching alone does not establish definitive student identity or scholarship-award authorization.

R9-C. Describe verification as an existing capability to refine

WHERE: Chapter V > Recommendations

FIND IN WORD: "The researchers recommend improving student-record verification"

Source body paragraph(s): 616. These are not printed page numbers.

WHAT TO DO: Replace this student-verification recommendation only.

STOP BEFORE: "The researchers also recommend the future integration"

WHAT TO PUT:

The researchers recommend further evaluating and refining the implemented student-record verification and duplicate-scholarship prevention mechanisms through additional testing using larger authorized datasets. Future improvements may focus on minimizing false-positive identity matches, strengthening the validation of incomplete student information, and improving the reliability of document extraction for low-quality or difficult-to-read submissions. These improvements are intended to strengthen existing validation capabilities rather than introduce duplicate prevention as an entirely new functionality.

R9-D. Distinguish current roster imports from future direct integration

WHERE: Chapter V > Recommendations

FIND IN WORD: "The researchers also recommend the future integration of external scholarship databases"

Source body paragraph(s): 617. These are not printed page numbers.

WHAT TO DO: Replace this external-integration paragraph only. Keep the following security/recovery recommendation and its honest limitations.

STOP BEFORE: "The researchers also recommend further strengthening"

WHAT TO PUT:

The researchers also recommend exploring future direct integration with authorized external scholarship databases and providers. Although the developed system already supports the inclusion of externally sourced scholarship information through official roster imports and authorized record management, direct system-to-system integration may improve the completeness, timeliness, and consistency of scholarship information.

Such integration would require appropriate institutional permissions, data-sharing agreements, security safeguards, and cooperation with participating scholarship providers. Future integration should complement the existing roster-import functionality while strengthening scholarship validation and monitoring capabilities.

R10-A. Conditional DSRM framework opening

WHERE: Chapter I > Conceptual Framework > opening paragraph

FIND IN WORD: "This study is anchored on the Input"

Source body paragraph(s): 64. These are not printed page numbers.

WHAT TO DO: ONLY IF the adviser approves DSRM: replace this opening paragraph. Otherwise keep the IPO framework and do not mark this conditional edit as an unfinished mandatory requirement. No graphic revision is included here.

WHAT TO PUT:

This study is guided by the Design Science Research Methodology (DSRM) proposed by Peffers et al. (2007), which provides a structured approach to developing and evaluating technological artifacts designed to address identified organizational problems. The methodology consists of six major activities: problem identification and motivation, definition of the objectives of a solution, design and development, demonstration, evaluation, and communication. These activities provide a research-oriented structure for explaining the development and evaluation of the centralized scholarship management and tracking system for Bulacan State University - Bustos Campus.

NOTE: Conditional text, not an instruction to change frameworks without approval. Do not claim a particular research method was followed unless its activities can be supported by the study record.

R10-C. Conditional written framework explanation

WHERE: Chapter I > Conceptual Framework > explanation after Figure 1.1

FIND IN WORD: "Figure 1.1 presents the conceptual framework of the study using the Input"

Source body paragraph(s): 71. These are not printed page numbers.

WHAT TO DO: ONLY IF DSRM is approved: replace the figure explanation and following IPO input/process/output explanation block, stopping before Scope and Limitations. Do not retain contradictory IPO and DSRM explanations. If this replacement is adopted, R1-B's old process paragraph is superseded rather than duplicated.

STOP BEFORE: "Scope and Limitations of the Study"

WHAT TO PUT:

Figure 1.1 presents the conceptual framework of the study based on the six major activities of the Design Science Research Methodology (DSRM).

Problem identification and motivation address fragmented scholarship records, manual application processing, monitoring difficulties, and conflicting scholarship-assignment concerns. Definition of the objectives of a solution establishes the need for centralization, validation, scholarship recommendation, monitoring, and reporting.

Design and development include the system architecture, database, user interfaces, weighted recommendation scoring and duplicate-detection workflows. Demonstration presents the developed registration, roster-import, assignment, approval and tracking functions. Evaluation assesses the system using functional and technical tests, TAM, and ISO/IEC 25010. Communication presents the study's findings, conclusions and recommendations.

Together, these activities explain the relationship between the identified scholarship-management problems, the developed technological solution, and the evaluation of that solution within Bulacan State University - Bustos Campus.

NOTE: DSRM is a methodology, not a conventional theory. Reconcile the Chapter III Developmental Research Design description and explain Agile as the development approach only if this reflects the research actually carried out. Do not silently rename the research design.

IF APPROVED, ADD THE CITED SOURCE TO REFERENCES: Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45-77. https://doi.org/10.2753/MIS0742-1222240302.

R10 consistency check. Locate Chapter III research-design wording

WHERE: Chapter III > Research Design and development methodology

FIND IN WORD: "This study employed a Developmental Research Design"

Source body paragraph(s): 210. These are not printed page numbers.

WHAT TO DO: If DSRM is approved, review this paragraph and the Agile methodology explanation with the adviser. No unconditional replacement is supplied because whether DSRM was actually followed cannot be established merely by editing the thesis.

WHAT TO PUT:

If approved and supported by the study record, distinguish the research methodology from the iterative development approach: DSRM structures the research activities, while Agile describes the iterative software-development process. Retain the existing design description when a change has not been approved.

R11-A correction. Use the algorithm actually implemented

WHERE: Chapter III > System Modules > 3. Scholarship Recommendation Module and 4. recommendation-mechanism subsection

FIND IN WORD: "3. Scholarship Recommendation Module"

Source body paragraph(s): 264. These are not printed page numbers.

WHAT TO DO: Replace the explanatory paragraphs under Module 3 and rename Module 4. Retain the module numbering. Replace the existing Decision Tree explanation under Module 4 and insert the code explanation and example below before 5. Validation Module. The source guide's Decision Tree terminology is outdated for this implementation.

STOP BEFORE: "5. Validation Module"

WHAT TO PUT:

3. Scholarship Recommendation Module

The Scholarship Recommendation Module helps students identify potentially suitable scholarship opportunities using available records and configured eligibility conditions. It uses Weighted Recommendation Scoring rather than a trained or rule-branch Decision Tree. Candidates must satisfy the implemented application-availability and GWA filters before receiving a ranking score. Recommendations support student decision-making and do not constitute scholarship approval.

4. Weighted Recommendation Scoring Mechanism

The ranking score combines grade fit, normalized scholar-roster popularity, location fit and announcement completeness. Grade fit contributes up to 40 points. Popularity contributes up to 22 points, increased to 30 for students with GWA of 1.75 or better. Location contributes up to 25 points. Announcement completeness contributes eight baseline points and two additional points when a supported image field is present. The resulting scores are sorted in descending order.

The current scoring function does not use family income, course, year level or achievements as weighted ranking factors. Application authorization and any other grantor-specific conditions remain separate from the recommendation score. The score is not an acceptance probability or measured accuracy percentage. The method was selected as a transparent ranking approach that can explain its factor contributions without a labeled historical training dataset. This rationale is not evidence of superiority over other algorithms.

R11-A. What to insert under Module 4

INSERT SUBHEADING: Implementation of the Weighted Scholarship Recommendation Algorithm

SUGGESTED INTRODUCTION: The following excerpt shows the implemented recommendation function. The function reads the student GWA and candidate records, excludes candidates that fail the application or grade conditions, calculates factor scores and sorts the resulting recommendations. It uses helper functions for configured scholarship policies, location scoring and display labels; it is not a stand-alone program.

ACTUAL SOURCE EXCERPT: backend/scholarship_rules.py, lines 137-216.

```
def recommend_scholarships(payload: dict[str, Any] | None = None) -> dict[str, Any]:
    payload = payload or {}
    student = payload.get("student") or {}
    scholarships = payload.get("scholarships") or payload.get("grantors") or payload.get("announcements")
or []
    student_gwa = _to_float(student.get("gwa") or student.get("currentGwa") or student.get(
"generalWeightedAverage"))
    max_roster = max([_to_int(item.get("rosterCount")) for item in scholarships] or [0])
    is_high_grade_student = student_gwa is not None and student_gwa <= 1.75
    recommendations = []

    for item in scholarships:
        if item.get("applicationsBlocked") is True:
            continue
        if item.get("applicationEnabled") is False and item.get("applyOpen") is not True:
            continue

        minimum_gwa = _to_float(item.get("minimumGwa") or item.get("minGwa") or item.get("minimumGrade"))
        provider = item.get("providerLabel") or item.get("grantorName") or item.get("title") or item.get(
"name") or ""
        if minimum_gwa is None:
            minimum_gwa = _to_float(get_scholarship_policy(provider)["minGwa"]) or 2.25
        if student_gwa is None or student_gwa > minimum_gwa:
            continue

        reasons = []
        grade_margin = max(0.0, minimum_gwa - student_gwa)
        grade_score = min(40.0, 24.0 + (grade_margin * 12.0))
        if grade_margin >= 0.5:
            reasons.append("Strong GWA match")
        else:
            reasons.append("Meets minimum GWA")

        roster_count = _to_int(item.get("rosterCount"))
        popularity_weight = 30.0 if is_high_grade_student else 22.0
        popularity_score = (roster_count / max_roster * popularity_weight) if max_roster > 0 else 0.0
        if roster_count > 0:
            reasons.append(f"{roster_count} scholar roster")

        location_score = _location_score(student, item)
        if location_score >= 22:
            reasons.append("Nearest location match")
        elif location_score >= 12:
            reasons.append("Same province area")
        elif _normalize_text(student.get("province")) == "pampanga" and _normalize_text(item.get(
"province")) == "bulacan":
            reasons.append("Bulacan grantor, lower location priority")

        completeness_score = 8.0
        if item.get("profileImageUrl") or item.get("authorImageUrl") or item.get("imageUrl"):
            completeness_score += 2.0

        score = grade_score + popularity_score + location_score + completeness_score
        label = _recommendation_label(score, reasons)
        recommendations.append(
            {
                "item": item,
                "score": round(score, 4),
                "label": label,
                "reasons": reasons[:4],
                "criteria": {
                    "gwa": student_gwa,
                    "minimumGwa": minimum_gwa,
                    "rosterCount": roster_count,
                    "gradeScore": round(grade_score, 2),
                    "popularityScore": round(popularity_score, 2),
                    "locationScore": round(location_score, 2),
                },
            }
        )

    recommendations.sort(key=lambda row: row["score"], reverse=True)
    return {
        "ok": True,
        "algorithm": "Weighted Recommendation Scoring",
        "weights": {
            "hardFilters": ["applications must be open", "student GWA must meet minimum GWA"],
```

```
            "gwaFit": "up to 40 points",
            "rosterPopularity": "up to 22 points, raised to 30 for high-grade students",
            "locationFit": "up to 25 points with a Pampanga-to-Bulacan priority reduction",
            "profileCompleteness": "up to 10 points",
        },
        "recommendations": recommendations,
    }
```

SUGGESTED EXPLANATION AFTER THE EXCERPT: The application flags and minimum-GWA comparison act as filters rather than learned decision-tree branches. The grade score rewards the margin by which the student's eligible GWA meets the minimum. Popularity is normalized against the largest roster count in the supplied candidate set, and the high-grade condition changes its maximum weight. Location and image availability add the remaining points. Sorting by the total creates the recommendation order. The same supplied inputs produce the same ranking, but changed candidate rosters or conditions may change the scores.

OPTIONAL SUPPORTING EXCERPT: include the location helper if the paper explains province/city/barangay points.

ACTUAL SOURCE EXCERPT: backend/scholarship_rules.py, lines 239-262.

```
def _location_score(student: dict[str, Any], grantor: dict[str, Any]) -> float:
    student_province = _normalize_text(student.get("province"))
    student_city = _normalize_text(student.get("city"))
    student_barangay = _normalize_text(student.get("barangay"))
    grantor_province = _normalize_text(grantor.get("province"))
    grantor_city = _normalize_text(grantor.get("city"))
    grantor_barangay = _normalize_text(grantor.get("barangay"))

    if not any([student_province, student_city, student_barangay, grantor_province, grantor_city,
grantor_barangay]):
        return 8.0

    score = 0.0
    if student_province and grantor_province and student_province == grantor_province:
        score += 10.0
    elif student_province == "pampanga" and grantor_province == "bulacan":
        score += 4.0
    elif grantor_province:
        score += 2.0

    if student_city and grantor_city and student_city == grantor_city:
        score += 10.0
    if student_barangay and grantor_barangay and student_barangay == grantor_barangay:
        score += 5.0
    return min(score, 25.0)
```

R11-A. Worked example using controlled inputs

WHAT TO PUT: Consider a controlled example with a student GWA of 1.50 and a Bulacan/Baliwag address. Scholarships A and B both specify a minimum GWA of 2.00, while C specifies 1.25. A has 20 roster entries, matches the student's province and city but not barangay, and has an image. B has 40 roster entries, is in Aurora/Baler, and has no image. Both A and B pass the supplied availability and grade conditions, while C is excluded because 1.50 does not meet its 1.25 requirement. The maximum candidate roster count is 40, and the high-grade popularity weight is 30.

Factor	Example A	Example B
Grade fit	24 + 12 x (2.00 - 1.50) = 30	30
Popularity	20 / 40 x 30 = 15	40 / 40 x 30 = 30
Location	Same province and city = 20	Other province, no city/barangay match = 2
Completeness	8 + image bonus 2 = 10	8, no image bonus
Total / order	75 / first	70 / second

WHAT TO PUT AFTER THE TABLE: Scholarship A ranks above B for these controlled inputs. These numbers were checked against the actual recommendation function. They demonstrate how the ranking works, not actual student outcomes, real grantor offerings or measured recommendation accuracy. If the adviser requires an actual program example, replace the fixture labels and input conditions with an authorized, verified real scholarship configuration and recompute all scores.

R11-A. Align other present-system descriptions

WHAT TO DO: Search for Decision Tree throughout the DOCX. Replace claims about THIS system with Weighted Recommendation Scoring and its actual filters/factors. Keep descriptions of Decision Trees genuinely used by cited studies. Do not globally replace the term inside research titles or references.

Where	Find in Word	What to change
Chapter II > Technical Background	In the context of this study, the developed system uses	Describe weighted ranking with application/GWA filters, not a Decision Tree.
Chapter II > literature comparisons and synthesis	The basic recommendation process involves	Retain the cited study's algorithm; correct only the comparison describing this system.
Chapter III > Back-End Technology	The back-end of the system was developed using Python	Replace the present-system algorithm name with Weighted Recommendation Scoring.
Chapter III > Scholarship Recommendation Mechanism	The system provides scholarship recommendations using	Use the same implemented factors and filters as Module 4.
Chapter III > development/prototyping discussion	The rule-based Decision Tree recommendation algorithm	Describe the implemented weighted recommendation and record-linkage mechanisms.
Chapter IV > Recommendation Module	The Scholarship Recommendation Module compares student information	State weighted scoring; recommendations are decision support, not approval.

R11-B. Add actual identity-matching and duplicate-detection excerpts

WHERE: Chapter III > System Modules > 6. Duplication Prevention Algorithm

FIND IN WORD: "When the system identifies a possible duplicate or conflict"

Source body paragraph(s): 295. These are not printed page numbers.

WHAT TO DO: Add the following subheading and introductory text after this paragraph and before 7. Approval Module. Keep the existing formulas, weight table and configured thresholds.

STOP BEFORE: "7. Approval Module"

WHAT TO PUT:

Implementation of the Record Linkage and Duplicate-Detection Algorithm

The duplicate-record checker first normalizes the available values to reduce differences caused by capitalization, punctuation and spacing. It calculates field-level similarities and combines available comparisons using configured weights. Missing comparisons are excluded and the remaining weights are renormalized. Name similarity considers both direct and token-sorted forms and retains the higher result.

The following excerpts show actual normalization, Levenshtein similarity, weighted comparison and matching conditions from the implementation. They are source excerpts, not an independent complete program.

ACTUAL SOURCE EXCERPT: backend/grantor_algorithms.py, lines 10-13.

```
def normalize_match_value(value: Any = "") -> str:
    normalized = normalize_space(str(value or "")).lower()
    normalized = re.sub(r"[^a-z0-9]+", " ", normalized)
```

```
    return normalize_space(normalized)
```

ACTUAL SOURCE EXCERPT: backend/grantor_algorithms.py, lines 16-17.

```
def normalize_identifier(value: Any = "") -> str:
    return re.sub(r"\s+", "", normalize_match_value(value))
```

ACTUAL SOURCE EXCERPT: backend/grantor_algorithms.py, lines 32-56.

```
def levenshtein_similarity(left_value: Any = "", right_value: Any = "") -> float:
    left = normalize_match_value(left_value)
    right = normalize_match_value(right_value)
    if not left and not right:
        return 1.0
    if not left or not right:
        return 0.0
    if left == right:
        return 1.0

    distances = list(range(len(right) + 1))
    for left_index, left_char in enumerate(left, start=1):
        diagonal = distances[0]
        distances[0] = left_index
        for right_index, right_char in enumerate(right, start=1):
            above = distances[right_index]
            substitution_cost = 0 if left_char == right_char else 1
            distances[right_index] = min(
                distances[right_index] + 1,
                distances[right_index - 1] + 1,
                diagonal + substitution_cost,
            )
            diagonal = above

    return 1 - distances[len(right)] / max(len(left), len(right))
```

ACTUAL SOURCE EXCERPT: backend/grantor_algorithms.py, lines 68-123.

```
def evaluate_scholar_duplicate(candidate: dict[str, Any] | None = None, existing: dict[str, Any] | None =
None) -> dict[str, Any]:
    candidate = candidate or {}
    existing = existing or {}
    candidate_name = scholar_full_name(candidate)
    existing_name = scholar_full_name(existing)
    direct_name_similarity = comparable_similarity(candidate_name, existing_name) or 0
    sorted_name_similarity = comparable_similarity(
        token_sorted_value(candidate_name),
        token_sorted_value(existing_name),
    ) or 0
    name_similarity = max(direct_name_similarity, sorted_name_similarity)
    candidate_student_id = normalize_identifier(
        candidate.get("studentId") or candidate.get("studentnumber") or candidate.get("studentNumber")
    )
    existing_student_id = normalize_identifier(
        existing.get("studentId") or existing.get("studentnumber") or existing.get("studentNumber")
    )
    candidate_email = normalize_identifier(candidate.get("email"))
    existing_email = normalize_identifier(existing.get("email"))
    candidate_phone = re.sub(r"\D", "", str(candidate.get("cpNumber") or candidate.get("contactNumber") or
""))
    existing_phone = re.sub(r"\D", "", str(existing.get("cpNumber") or existing.get("contactNumber") or ""))
    exact_student_id = bool(candidate_student_id and candidate_student_id == existing_student_id)
    exact_email = bool(candidate_email and candidate_email == existing_email)
    exact_phone = bool(candidate_phone and candidate_phone == existing_phone)
    fields = [
        {"label": "student ID", "weight": 0.32, "value": comparable_similarity(candidate_student_id,
existing_student_id, True)},
        {"label": "name", "weight": 0.30, "value": name_similarity or None},
        {"label": "email", "weight": 0.10, "value": comparable_similarity(candidate_email, existing_email,
True)},
        {"label": "contact number", "weight": 0.08, "value": comparable_similarity(candidate_phone,
existing_phone, True)},
        {"label": "course", "weight": 0.08, "value": comparable_similarity(candidate.get("course"),
existing.get("course"))},
        {"label": "year level", "weight": 0.04, "value": comparable_similarity(candidate.get("yearLevel")
or candidate.get("year"), existing.get("yearLevel") or existing.get("year"))},
        {"label": "city", "weight": 0.04, "value": comparable_similarity(candidate.get("city"), existing.
get("city"))},
```

```
        {"label": "province", "weight": 0.04, "value": comparable_similarity(candidate.get("province"),
existing.get("province"))},
    ]
    compared_fields = [field for field in fields if field["value"] is not None]
    compared_weight = sum(field["weight"] for field in compared_fields)
    weighted_score = (
        sum(float(field["value"]) * field["weight"] for field in compared_fields) / compared_weight
        if compared_weight > 0
        else 0
    )
    strong_identifier_match = exact_student_id or ((exact_email or exact_phone) and name_similarity >= 0.72)
    is_duplicate = strong_identifier_match or (name_similarity >= 0.82 and weighted_score >= 0.84)
    reasons = [field["label"] for field in compared_fields if float(field["value"]) >= 0.9]

    return {
        "isDuplicate": is_duplicate,
        "score": weighted_score,
        "nameSimilarity": name_similarity,
        "reasons": reasons,
        "exactStudentId": exact_student_id,
        "exactEmail": exact_email,
        "exactPhone": exact_phone,
        "algorithm": "Weighted Record Linkage with Levenshtein Similarity",
        "fields": compared_fields,
    }
```

SUGGESTED EXPLANATION: Levenshtein similarity is calculated as one minus the edit distance divided by the longer normalized string length. The weighted score combines available fields rather than comparing names alone. An exact Student ID may identify a potential duplicate independently. Exact email or contact number requires at least 72% name similarity; the general fuzzy rule requires at least 82% name similarity and 84% weighted similarity. These are configured thresholds, not measured accuracy percentages. A similarity flag is not proof of an existing scholarship award.

SUGGESTED TRANSITION TO ROSTER ASSIGNMENT: Official scholarship assignment applies a different decision process. The authoritative transaction checks the exact student identifier, normalized full name, verified academic cycle, active account, recognized program and existing commitments. It does not use a fuzzy-name match alone to authorize another award. The following excerpts show selected guards from the transaction; the complete function also handles application conversion, slot release, preservation of records and audit/notification effects.

COMPLETE SOURCE FUNCTION: public.assign_authoritative_roster_scholarship in the migration named below. The next three excerpts are selected portions of that function.

ACTUAL SOURCE EXCERPT: supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql, lines 88-97.

```
  select data into roster_data from public.grantor_portal_scholars where id = p_roster_id and parent_id =
p_grantor_id for update;
  if not found or coalesce(roster_data->>'studentId', roster_data->>'studentnumber', roster_data-
>>'studentNumber', '') <> p_student_id
    or not public.roster_record_active(roster_data) then raise exception 'roster_match_not_found'; end if;
  -- Untagged legacy rows require staff verification; never guess their semester.
  if public.roster_cycle(roster_data) <> cycle then raise exception 'roster_cycle_requires_verification';
end if;
  if public.roster_name_key(coalesce(roster_data->>'fullName', concat_ws(' ', roster_data->>'fname',
roster_data->>'mname', roster_data->>'lname'))) = ''
    or public.roster_name_key(coalesce(roster_data->>'fullName', concat_ws(' ', roster_data->>'fname',
roster_data->>'mname', roster_data->>'lname')))
      <> public.roster_name_key(coalesce(student_data->>'fullName', concat_ws(' ', student_data->>'fname',
student_data->>'mname', student_data->>'lname'))) then
    raise exception 'roster_identity_mismatch';
  end if;
```

ACTUAL SOURCE EXCERPT: supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql, lines 115-125.

```
  commitment_id := nullif(state_data#>>'{scholarshipCommitment,applicationId}', '');
  if commitment_id is not null and commitment_id <> application_id then
    select data into commitment_data from public.scholarship_applications where id = commitment_id;
    if (commitment_data is null and public.roster_cycle(state_data->'scholarshipCommitment') in ('', cycle))
      or (commitment_data is not null and public.roster_cycle(commitment_data) in ('', cycle)
```

```
        and public.roster_record_active(commitment_data)) then raise exception
 'scholarship_already_committed'; end if;
   end if;
   if exists(select 1 from public.scholarship_applications where data->>'studentId' = p_student_id and id <>
 application_id
     and public.roster_cycle(data) in ('', cycle) and public.roster_application_awarded(data)) then
     raise exception 'scholarship_already_committed';
   end if;
```

ACTUAL SOURCE EXCERPT: supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql, lines 131-138.

```
   -- An existing award survives grantor archival under office management, but
   -- an inactive provider/program cannot authorize a new assignment.
   if not exists(select 1 from public.grantor_portal_announcements where parent_id = p_grantor_id
     and (id = target_scholarship_id or data->>'programId' = target_scholarship_id or data->>'scholarshipId'
 = target_scholarship_id)
     and public.roster_record_active(data)) then raise exception 'roster_scholarship_not_recognized'; end if;
   if not exists(select 1 from public.providers where id = p_grantor_id and public.roster_record_active(
 data)) then
     raise exception 'roster_match_not_found';
   end if;
```

SUGGESTED EXPLANATION AFTER THE EXCERPTS: These guards distinguish identifying a possible duplicate from confirming an official scholarship assignment. A mismatched identity, an unverified cycle, an unrecognized program or a conflicting award prevents automatic assignment. Records requiring resolution are handled through authorized administrative procedures rather than being automatically awarded through a high similarity score.

Code is taken from current local source files, not from a live deployment. Helper functions and surrounding transaction statements are intentionally not all reproduced. PDF line wrapping is for readability; copy the exact code from the named source file if indentation or long lines are affected. Retain source paths/function names as traceability notes in the thesis or appendix.

R12-D. Introduce and caption the new technical tables

WHERE: Chapter III Testing or Chapter IV Objective No. 2 results, immediately above each new technical test table

FIND IN WORD: "The Validation Module supports student-record verification"

Source body paragraph(s): 491. These are not printed page numbers.

WHAT TO DO: Use this anchor to locate the R7-A insertion area. Add an introductory paragraph and a numbered caption above each new completed technical-test table. Do not duplicate the intro already supplied in R7-A. The general wording below can be adapted for another completed technical table.

WHAT TO PUT:

Table [assign number] presents the selected test cases used to evaluate the corresponding system functionality. The table identifies the test-case reference, expected behavior, actual observed result and corresponding status. These results provide supporting technical evidence for the implemented features under the documented testing conditions.

Table [assign number]. [Specific Title Describing the Tested Function]

NOTE: For document-extraction results, use the actual sample/reference/extracted-value columns rather than pretending the template worksheet is a completed test table. Update table numbers and cross-references after all insertions. Image layout and figure replacement are excluded from this text guide.

Final Editing Order and Checks

- Complete R1-A/B/C; keep the already completed R2/R2-B and approval/roster explanations.

- Insert the R5 status table and finish R6/R6-B technical-testing context.

- Add R7-A results with the verified selected Solis cases, then replace the R7-B written explanation.

- Insert the proposed R8-A method and the honest R8-B limitation. Obtain authorized samples and run the evaluation before adding measured accuracy results.

- Replace the four specified Chapter V paragraphs using R9-A/B/C/D.

- Apply R10-A/C only after adviser approval; reconcile research-design wording rather than silently changing the methodology.

- Complete R11 using actual source excerpts and the checked worked example. Correct present-system Decision Tree claims, but retain accurate related-literature citations.

- Add R12-D introductions/captions where required. Replace table-number placeholders only after the final numbering is known.

- Remove instruction labels, pending-result templates and unfilled number placeholders from final completed-results sections. Keep unresolved evidence honestly described as a limitation.

- Do not change the TAM/ISO respondent counts or statistical results merely because technical test evidence has been added. Re-export the edited thesis to PDF and check its final contents and references.

Only missingrevision.pdf is regenerated by this script. The DOCX manuscript and the original Solis guide remain unchanged.